

Lab Assignment of Architecture of Distributed Systems Q1 2026-2027

Implementing and Evaluating Word Count Service

This lab assignment introduces how an architectural style can be used to design distributed systems that process large volumes of data across multiple nodes. The assignment provides an opportunity to apply concepts covered in the lectures, such as interaction styles, scalability, load balancing, and fault tolerance.

In this assignment, you will design, develop, implement, and test a word count service that provides real-time counting of *words* from a piece of text. Each client can request the occurrence of a specific keyword in a text file. The system should accurately provide the number of occurrences of the specified keyword in each request. Additionally, the system should cache the answer to each client request in an in-memory database to reduce execution latency for identical future requests. The architecture should support:

- Clients sending requests with a **keyword** and a reference to a **text** stored on the server.
- The server processing the request and returning the count of the keyword in the given text.
- Caching the results in an in-memory database (e.g., **Redis** [2]) to support additional queries, such as tracking the most frequently requested keywords (hot keywords).

Word counting has many applications in distributed system services, such as text analysis and plagiarism detection.

Objective

At the end of the lab assignment,

1. **Architectural Modeling:** You will learn how to create a basic architectural model for a distributed system to offer functional and non-functional requirements. This will be achieved by identifying the system's building blocks or components and connecting them.
2. **Performance Evaluation:** You will learn how to check the performance of the system and understand the need for scaling a single server-based architecture. This will be done by evaluating performance metrics relevant to the system. For instance, in the case of the word count service, you will measure the execution latency (see the definition in Phase 1) for each request, based on a time unit, such as milliseconds.
3. **Load Balancing:** You will practice the load-balancing aspect of the system by understanding the need to scale the number of available servers. You will replicate your server and develop a load balancer to distribute incoming requests across a set of available servers.
4. **Fault Tolerance:** You will practice the fault tolerance aspect of the system. By testing the functionality of your system, you will learn how to identify a failed server and how to implement measures to ensure that the system remains reliable and resilient after failure.

Tools & Libraries

You will use Docker Compose [1] to build the virtualized version of your distributed system to offer the envisioned word count service. The Python programming language should be used to implement the required system functionalities.

To start with the tools, download Docker Desktop from <https://www.docker.com/products/docker-desktop/> and install it. Docker Desktop is available on most operating systems, including Windows, Linux, and Mac. You do not need to install Python on your machine that runs Docker Desktop, as the developed code for the client, server, and load balancer will be executed inside the corresponding containers created via Docker. You can refer to the assignment's description of *Phase 1* for more details. We now briefly explain docker-compose and the rpyc library.

Docker Compose. Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration. We provide more details while describing the tasks you need to perform with docker-compose.

RPyC. RPyC (pronounced as are-pie-see), or Remote Python Call, is a transparent Python library for symmetrical remote procedure calls (RPC), clustering, and distributed computing. RPyC uses object-proxying, a technique that employs Python's dynamic nature, to overcome the physical boundaries between processes and computers so that remote objects can be manipulated as if they were local. Note that, unlike many other RPC implementations that rely on protocols like HTTP for communication, RPyC relies on transport layer protocols to reduce the overhead of message passing.

We provide some hints and tips that help you develop the lab assignment faster by saving time in getting basic knowledge about the tools and systems. You will find our [hints inside green boxes](#) and [tips within violet boxes](#). You can ignore them if you already have enough experience using them.

1 Assignment

The assignment consists of four phases, each with tasks that you need to carry out. You will write a report showing how different aspects of the system are designed, developed, and tested.

Phase 1: Architectural Model

The main objective of this phase is to design the overall architecture of your distributed service. You are asked to design a system where clients request the number of occurrences of a keyword in a text document that is stored on the server.

1. **Requirements & Stakeholders:** Provide two functional and two non-functional requirements of your system. Identify two stakeholders of the system and explain their roles in the system.
2. **Architectural Diagram & Description:** Include a clear diagram that shows the architecture of your system. Your service must follow a [Client-Server](#) architectural style. Your diagram should illustrate the key components and the connectors between them. If needed, you may explain the main functionality of the components and connectors concisely.
3. **Architectural Style Trade-off Analysis:** While your implementation must follow the [Client-Server](#) model, briefly analyze the suitability of two other architectural styles, i.e., [Peer-to-Peer](#), [Layered](#), and [Publish-Subscribe](#), covered in the course. Discuss whether and how each could be used to develop a similar service.

Phase 2: Implementation of the Service

1. Implement the architecture using virtualization techniques. Use Docker for virtualization, since it is a lightweight tool that enables large-scale virtualization with limited physical resources. For more information, see the [Supplementary learning hub page on Canvas](#) or the Docker online documentation. Docker is compatible with most current operating systems, including Windows, Mac (including those based on Arm architecture, e.g., M1 and M2), and Linux. You can check the Docker documentation on how to create a Docker cluster with docker-compose at <https://docs.docker.com/compose/gettingstarted/> or check the ebook via the TUE library in [\[4\]](#).

How to create a docker image and a docker container

Open a terminal in your operating system to interact with Docker. Depending on your system's configuration, you might need to run Docker commands with `sudo` for proper permissions. If you encounter any permission-related errors, try adding `sudo` at the

beginning of the command. For example, instead of running `docker run ...`, use `sudo docker run ...`.

To create images that can constitute the template for a class of containers impersonating the same role (e.g., a client or a server), you can proceed as follows. Note that you need to carry out this step only once (for each image you want to generate, so, for example, once for all the clients and servers of the system).

Create an empty directory (suppose it is called "wordCount"). Inside that directory, create a file named "Dockerfile" with the following contents:

```
FROM python:3.9-slim-buster
RUN apt-get update
#RUN apt-get install -y "any package you like"
RUN pip3 install rpyc
CMD [ "/bin/bash", "-c", "while true; do bash -l; done" ]
```

These instruct Docker to create the image starting from a `python:3.9-slim-buster` distribution and installing a set of basic packages inside it. They also specify which command (CMD) should be executed by a container when it uses this image. Essentially, in this case, we run a shell that never exits (so that you can never quit the container by accident). At this point, enter the "wordCount" directory and run the image build command (don't forget the final '.'):

```
$ docker build --pull -t ads:lab .
```

"ads:lab" is the name of the image and has to be all lowercase and follows 'imageName:Tag'. You will see a lot of messages confirming the ongoing installation of the required packages inside the image. At this point, your "template" is ready to use, and it already contains the required software packages to create your cluster. You can easily find it inside the list of images:

```
$ docker image ls
```

Now, you can create as many containers as you need from this image. Since creating a cluster with many containers can be time-consuming and error-prone, we will use docker-compose to do these tasks automatically. Please check [1] for more details.

You can start your cluster after preparing the `docker-compose.yml` file as follows.

```
$ docker-compose up
```

Then, you can check the number of images docker created for you by running `docker image ls` and containers by running `docker ps -a` in your terminal. If you end up with a large list of images that are called dangling images, use the following command to remove them.

```
$ docker image prune
```

2. Develop and test the `rpyc` RPC server and client for the word count service. The server should have the capability of answering client's requests and caching the results in Redis as mentioned in item 1 of Phase 1. You can use a dedicated container to run Redis.

💡 How to implement the service

- As example texts to count in, you can use your desired movie scripts taken from <https://imdb.com/> or free books from Project Gutenberg <https://www.gutenberg.org/>. Other sources of text are welcome as long as they are free to use.
- You can check the `rpyc` documentation on how to program with it at <https://rpyc.readthedocs.io/en/latest/docs.html>.
- The official documentation of using Redis with Python can be found in <https://docs.objectrocket.com/redis/python/examples.html>

3. You should demonstrate in your report that the server can respond to client requests by measuring the execution latency of each request on the server. The **execution latency** is defined as the time difference between sending a request from the client to the server and receiving the response to that request from the server on the client side. Create two figures showing the server's latency performance when processing workloads at 5 different keyword request rates. Make sure the lowest rate is at least 10 requests per second, and the interval between rates is at least 10. For example: 50, 70, 90, 110, and 130 keyword requests per second. The first figure should present the average execution latency, while the second figure should present the 99th-percentile (tail) latency. The x-axis should represent the number of keyword requests (50–130 based on the example above), and the y-axis should represent the request execution latency (ms). You may choose any suitable visualization (e.g., line chart or bar chart) to illustrate the server's performance. For more information about tail latency, see [3].

Tips:

Note that, depending on your machine, and since the service runs in a virtualized environment, achieving even 100 requests per second might be difficult. In this case, you may use rates below 100 requests per second. However, we highly recommend using higher rates with larger intervals if your machine can support it.

Phase 3: Scalability & Load Balancing

This phase of the lab assignment assesses the scalability aspect of the system. Since the initial architecture relies on a single server, the system will not scale when it receives a large number of requests. Therefore, you need to replicate the servers and balance the load among them. Please replicate the server twice, so that you will have 3 servers.

4. Update the figure for the architecture of your system in Phase 1 to include the load balancer and replicas.
5. Create a docker container and implement a load balancer in a way that it can distribute the load based on **two** well-known **dynamic** load-balancing algorithms. You can use Python to develop the load balancer.

💡 How to balance the load dynamically

To add load balancing to the Docker service, you need to develop a Docker container that distributes the incoming service requests to the available servers. Load balancing can be done statically (pre-configured without considering the server load or any other infrastructure-related resources) or dynamically (considering the load of servers before distributing the incoming traffic). Here is a list of some dynamic load-balancing algorithms (<https://aws.amazon.com/what-is/load-balancing/>):

- Least Connections or weighted Least Connections
- Least response time
- Resource-based

The load balancer should function like a network switch— forwarding client requests to one of the servers based on the load-balancing algorithm. ***The load balancer should not implement this using Remote Procedure Calls as done in RPyC***, because this would be way too inefficient. Instead, you can use network sockets (***not*** WebSocket) to receive and send messages (requests and responses), as well as multithreading or concurrency to handle multiple requests. The client and servers should communicate via RPyC, while the load balancer handles their interactions at the byte-stream level. The **Asyncio** library (<https://docs.python.org/3/library/asyncio.html>) should prove useful for this.

You should consider possible ways to test the scalability of the system by distributing the incoming requests to the available servers. Then, you need to demonstrate that the requests

reach the targeted servers through the load balancer. To show that the system does not scale, one option is to measure the processing time of the request for a batch of requests you send. One way to do this is to show the address or the name of the server serving the requests. Of course, you need to use your creativity for more ideas.

6. Use the client to send requests to the server and observe how the load is balanced across the servers in the cluster.

Hints

To avoid naming collisions among multiple servers and the need for additional components to manage names and identifiers, we assume the load balancer is part of the server environment, acting as a proxy for this assignment. To simplify the implementation, requests should be sent to the load balancer, which then distributes them according to the load-balancing algorithm.

7. **Deploy the service using Docker compose:** You should build and deploy your cluster using Docker compose. You can also specify each container's code as the client and server must run in the `docker-compose.yml` file.
8. **Test the load balancing:** You should test the load balancing of your service by sending requests to the service and monitoring the distribution of requests across the different servers in the cluster. You should test two of the load-balancing algorithms listed above. You need to provide a well-reasoned argument in your report about the distribution of the requests to the available servers based on the implemented load-balancer algorithms. Specifically, you should:
 - (a) Provide a screenshot from the Docker terminal showing that the requests are distributed to the available replicas and explain that they are distributed correctly according to the load-balancing algorithm.
 - (b) Present a comparison of the execution latency for executing the same requests with one server in Phase 2 and three servers in Phase 3, for both load-balancing algorithms. Explain the differences in execution latency.

Tips:

- (a) For replicating a server, you would need to modify the `docker-compose.yml` to include multiple replicas of the service and configure the load balancer to distribute the incoming requests to the replicas.
- (b) Please do not use the built-in replication feature of Docker Compose that creates replicas automatically. This feature assigns container's name randomly and has a built-in load balancer.
- (c) You may need to create a network to balance the load, and if the network already exists, you can remove it by using the following command.

```
$ docker network prune
```

- (d) You may need to remove the current images of the cluster that docker-compose made for you, since the updated code is not producing the intended output; you can remove the images using the following command.

```
$ docker rmi -f IMAGE-NAME
```

Phase 4: Load Balancing with Fault Tolerance

9. Introduce a failure in one of the servers (such as shutting down the server) and observe what happens in the system. Does the load still get balanced correctly? Does the system keep running at all?
10. Implement the capability of health monitoring of replicas in the load balancer by detecting failed servers and re-routing requests to healthy servers. You should maintain the health status of the

servers within the load balancer when distributing the requests. This helps in identifying the failed servers in your Docker cluster.

11. Test the fault-tolerance mechanism by introducing a failure in one of the servers and verifying that requests are re-routed to healthy servers. You should show the status of the servers using docker as evidence of fault tolerance. The load balancer should periodically perform health checks on the available servers. If the failed server comes back online, the load balancer should be able to distribute requests to the recovered server again.

💡 The last thing you need to know is how to stop/fail a running Docker container. You can check the Docker documentation for more details.

2 Deliverables

We describe the report requirements concerning technical content in the respective phase sections. Every group is required to write and submit a report using an IEEE LaTeX double-column template (<https://www.ieee.org/conferences/publishing/templates>). The report must be in PDF format and at most 3 pages of text; figures and tables do not count towards the page limit. The report should include a section for each phase. Make sure all figures are readable without the need for excessive zooming. You are not allowed to make any template style changes (e.g., margins, paragraph denotation, etc.). Copying work or parts of a work, whether work from the internet or work from other groups, will be considered fraud.



Your report should include the output of each phase separately using screenshots wherever needed. Make sure the text in the screenshots is readable without excessive zooming.

2.1 Phase 1 (1.5 points)

For this phase, the report should contain:

1. The requirements, both functional and non-functional, relevant stakeholders, and their roles.
2. The architecture of the system with a figure.
3. Argumentation for the choice of two architectural styles.

Rubric

	No points	Half points	Full points
Requirements and stakeholders (0.5pts)	The requirements are not relevant, or the stakeholders are not identified correctly.	Some requirements are not relevant or correct, or some stakeholders' roles are not discussed clearly.	All requirements are relevant and correct, and two relevant stakeholders are identified with their roles properly explained.
Architecture (0.5pts)	Many relevant elements (building blocks, connectors, etc) are missing from the architecture diagram or no description is given.	Some relevant elements are missing, or the description of the diagram is somewhat unclear.	All relevant elements are in the architecture diagram, and the description is clear.
Architectural styles (0.5pts)	The argumentation for the architectural styles is unclear or incorrect.	The argumentation for the architectural styles is somewhat unclear.	The argumentation for both architectural styles is clear.

2.2 Phase 2 (1 point)

For this phase, the report should contain:

1. A section in the report that defines the exact experiments ran for Phase 2 part 3, and that shows and explains the results of these experiments.



Additional submission: a zip file *lab-groupID-phase2.zip* containing the code of your implementation of Phase 2.

Rubric

	No points	Half points	Full points
Report (1pt)	Multiple aspects are unclear or missing.	The experiments ran are unclear, or there are less experiments ran than asked for, or the results are somewhat unclear, or the explanation of the results does not make sense.	The experiments ran are clear, the right number of experiments are run, the results are clearly visualised and the explanation of the results makes sense.

2.3 Phase 3 (3.5 points)

For this phase, the report should contain:

1. The updated architecture of the system from Phase 1 by adding the load balancer, with a brief description of the architecture in your report.
2. Argumentation for the need for a load balancer.
3. Details on your use and analysis of the load-balancing algorithms chosen:
 - High level explanation of the first load-balancing algorithm and how it is implemented.
 - High level explanation of the second load-balancing algorithm and how it is implemented.
 - Comparison of the load-balancing algorithms according to Phase 3 part 8.



Additional submission: a zip file *lab-groupID-phase3.zip* containing the code of your implementation of Phase 3.

Rubric

	No points	Half points	Full points
Architecture (0.75pts)	Many relevant elements are missing from the architecture diagram or no description is given.	Some relevant elements are missing, or the description of the diagram is somewhat unclear.	All relevant elements are in the architecture diagram, and the description is clear.
Need for load balancer (0.75pts)	The argumentation is unclear or incorrect.	The argumentation is somewhat unclear.	The argumentation is clear and correct.
First load-balancing algorithm (0.5pts)	The explanation of the first load-balancing algorithm or its implementation is unclear or missing, or the load-balancing algorithm is a static load-balancing algorithm.	The explanation of the first load-balancing algorithm or its implementation is somewhat unclear.	The explanation of the first load-balancing algorithm and its implementation is clear.
Second load-balancing algorithm (0.5pts)	The explanation of the second load-balancing algorithm or its implementation is unclear or missing, or the load-balancing algorithm is a static load-balancing algorithm.	The explanation of the second load-balancing algorithm or its implementation is somewhat unclear.	The explanation of the second load-balancing algorithm and its implementation is clear.
Comparison (1pt)	The experiments run to compare the two load-balancing algorithms, the results, or the conclusions taken from their results are unclear or missing.	The experiments run to compare the two load-balancing algorithms or the conclusions taken from their results are somewhat unclear.	The experiments run to compare the two load-balancing algorithms are clearly presented, the results are clearly presented and the conclusions taken from the results are relevant and clearly presented.

2.4 Phase 4 (2.5 points)

For this phase, the report should contain:

1. The results for Phase 4 part 9, that show what happens in your implementation for Phase 3 if you introduce a failure in one of the servers, along with a motivation for the need of a failure detection mechanism based on these results.
2. Explanation of the failure detection mechanism implemented in your system.
3. Results of experiments that show that your implementation for Phase 4 correctly balances the load between all active servers after introducing a failure in a server, and also after restarting the failed server, for both load-balancing algorithms.



Additional submission: a zip file *lab-groupID-phase4.zip* containing the code of your implementation of Phase 4.

Rubric

	No points	Half points	Full points
Failure before fault tolerance (0.5pts)	The results are unclear or the motivation for fault tolerance is missing.	The results of the experiment do not clearly show a failure, or it is not really clear why there is not failure, or the motivation for fault-tolerance is somewhat unclear.	The results of the experiment clearly present an issue in the load introducing a failure in a server and the reason for having fault-tolerance is well motivated.
Explanation of failure detection mechanism (1pt)	The explanation of the fault-tolerance mechanism is unclear or a significant part of the mechanism is missing.	The explanation of the fault-tolerance mechanism is somewhat unclear or a part of the mechanism is missing.	The fault-tolerance mechanism is clearly explained and complete.
Fault tolerance first load-balancing algorithm (0.5pts)	The presentation or explanation of the change in behaviour when introducing faults for the first load-balancing algorithm is unclear or missing.	The presentation or explanation of the change in behaviour when introducing faults for the first load-balancing algorithm is somewhat unclear, or the change in behaviour when reactivating a server is missing.	The change in behaviour when introducing faults and reactivating servers for the first load-balancing algorithm is clearly presented and explained.
Fault tolerance second load-balancing algorithm (0.5pts)	The presentation or explanation of the change in behaviour when introducing faults for the second load-balancing algorithm is unclear or missing.	The presentation or explanation of the change in behaviour when introducing faults for the second load-balancing algorithm is somewhat unclear, or the change in behaviour when reactivating a server is missing.	The change in behaviour when introducing faults and reactivating servers for the second load-balancing algorithm is clearly presented and explained.

2.5 All phases (1.5 points)

For any phase that involves implementation (2, 3 and 4), you need to submit your code in a zip file alongside the report. At the end, there is a debrief session where you show the functionality of your code of all three implementation phases, which you are expected to attend.

Rubric

	No points	Half points	Full points
Code (0.5pts)	Not all code was submitted.	-	All relevant code was submitted.
Debrief session (graded individually) (1pt)	The debrief session was not attended or the student did not seem to understand the groups solution at all.	The debrief session was attended, but the student did not seem to know the group's solution that well.	The debrief session was attended and the student seems to know the group's solution well.

3 Questions and Answers

In case there are questions or uncertainties regarding the lab assignment, and no answers have been provided in the manual and/or on Canvas, please post a message in the discussions section on Canvas. The lab staff will respond to questions and clarify uncertainties as frequently as possible. Emails regarding the lab assignment will not be answered, as emails do not help other student groups that might have similar questions or face the same uncertainties.

References

- [1] Docker Compose. <https://docs.docker.com/compose/>, 2023.
- [2] Redis - The Real-time Data Platform. <https://redis.io/>, 2024.
- [3] Jeffrey Dean and Luiz Andre Barroso. The tail at scale. Commun. ACM, 56(2):74–80, 2013. <https://doi.org/10.1145/2408776.2408794>.
- [4] Nigel Poulton. Docker Deep Dive. Packt Publishing, 2020. <https://tue.on.worldcat.org/v2/search?queryString=Docker%20Deep%20Dive>.